



PROTOCOL

ANATOMY OF A PRIVATE SUBSCRIBE TX

Who acts on this transaction, and how each one hides what needs to be hidden.

On May 23, 2026 at 17:05 GMT+1, a user subscribed to a private service through Protocol 01. The Solana transaction is public, finalized. Yet no observer (not the merchant, not an attacker, not even the relayer itself) knows **who** paid, **how much**, or **for which service**. Here is how.

Reference tx

2sQJzSreFhtMZz9Y3HkYpfAFgLRNqPoDfkjFk8kpFhUnYXAiENTmXyY3fgjBGVGM5vpDDbTvbtxVu3WeRSQ7DeqR

Devnet · Slot 464,392,604 · May 23, 2026 17:05:56 GMT+1 · Status: Success, Finalized

6

ON-CHAIN ACTORS

1

INSTRUCTION

18,533

COMPUTE UNITS

0.000005

SOL FEE

What to remember

Only one instruction is visible: **P01Relayer: SubmitJob**. Everything else (the STARK proof, the note commitment, the merchant identity, the subscriber identity) is **encrypted in a Base64 payload** that only the assigned relayer can decrypt, just-in-time. Nobody else can ever read it.

01 Phase 0 • Confidential relay

This tx is the **entry point** of the protocol. The subscribe itself (with its 80 KB STARK proof) will be executed **later, by an assigned keeper**, in a second tx that nobody can link back to you.

- 1 Mobile generates 2 STARK proofs on-device**
Circuit 0 (subscriber ownership) + Circuit 1 (pool commitment). About 12 s + 50 s on a Nothing A001. The spending key never leaves the phone.
- 2 Mobile builds the full subscribe tx (~1.5 KB)**
Discriminator `subscribe_private_stark`, merchant pubkey, fee escrow, STARK proof buffer reference, vk_hash subscriber. This tx is **never** published in cleartext.
- 3 Mobile encrypts the tx for the relayer (ML-KEM 768 + AES-GCM)**
The encrypted payload (~1.4 KB) is readable **only** by the keypair of the assigned relayer. If the relayer loses its key, nobody can recover it.
- 4 Ephemeral submitter POSTs to p01_relayer • THIS IS THE TX WE ANALYZE**
The submitter is a keypair generated on the fly for this tx only. It pays 0.000005 SOL of gas plus deposits 0.01 SOL into fee escrow for the keeper.
- 5 Keeper DEjW41... decrypts the payload**
The keeper sees the subscribe tx in cleartext for **a few milliseconds**, just long enough to submit it on-chain. It does not know **who** is behind the ephemeral submitter.
- 6 Keeper submits the real subscribe tx (2nd tx, within 1-2 min)**
This second tx contains the on-chain STARK verify plus the opening of the SubscriberVault. The signer of this tx is the **keeper**, not you. No graph link between your wallet and the subscribe.

The key insight: this tx does not contain the subscribe. It contains an **encrypted order** to subscribe. The subscribe itself is **decoupled in time and in signer** from your action.

02 Who signs, who pays, who holds the payload

1 Submitter

SIGNER • FEE PAYER • WRITABLE

94eNVrzobxDdrYkuukDkP2af8Udoo3TmWeYL8Cv7Rp3Z

Ephemeral keypair generated on the mobile, **never reused**. Receives just enough SOL from the user wallet to pay this tx (0.02 SOL), then burns. Off-chain, nothing ties it back to the user.

What it HIDES: the user wallet identity. An on-chain observer sees 94eNVr... as the gas payer but cannot trace back to the user wallet that funded the submitter (the funding can flow through a mixer, an exchange withdrawal, etc).

What it does NOT prevent: funding correlation. If the submitter is funded directly from your wallet in one hop, a graph analyzer can trace back. Mitigation: stealth funding via a Solana Pay link, or pre-funding through a mixer.

2 Job PDA

WRITABLE • CREATED IN THIS TX • 1419 BYTES

2jf5iVZWjrNNghjDnQMH7A6jh2JAsaQ9ZUeCwPVLohno

PDA derived from ["relay_job", job_id] where job_id is a random 32-byte nonce generated on the mobile. Stores the encrypted payload (~1.4 KB) plus routing metadata.

What it HIDES: the content of the subscribe tx. The **Encrypted Tx** field is ~1.4 KB of unreadable base64 (ML-KEM 768 + AES-GCM, post-quantum). Nobody outside the assigned relayer can decrypt it, ever.

What it does NOT prevent: the payload size is observable (1419 bytes). An indexer can know **that** a privacy operation is happening, and roughly its size. Mitigation: uniform padding (Phase C v1) on STARK upload, to be extended to the relayer in Phase E.

3 Assigned Relayer (Keeper PDA)

WRITABLE • STATE UPDATE

DEjW41JAmTLTq5BwtNRBzvBwPL3RxPXPEhvyYLuyxdBR

PDA of the keeper assigned in the p01_relayer registry. Its ML-KEM keypair is published at registration time; this is the **public key** the mobile used to encrypt the payload in step 3.

What it HIDES: the link between user wallet and subscribe tx. The keeper will sign the second tx (subscribe + STARK verify) on your behalf. On Solana, an observer sees "**the keeper subscribed someone**" and that is all. The keeper-side job queue introduces a non-deterministic delay (1 to 120 s), breaking timing correlation.

Trust assumption: the keeper sees the payload in cleartext for a few ms during submission. Current mitigation: multi-relayer registry, the mobile picks randomly among N active keepers. Future mitigation (Phase E): Intel SGX/TDX TEE so even the keeper never sees the cleartext.

03 Who orchestrates, who validates, who transfers

4 Config PDA

WRITABLE · 128 BYTES · REGISTRY STATE

G4eABk8c8pAerK28ECvnXwGVmvUSQQVaERw3UL1B4kVZ

Singleton PDA of the p01_relayer program. Holds the list of registered keepers (their PDAs plus ML-KEM public keys plus last_active_slot for failover) and fee parameters.

What it HIDES indirectly: the identity of the **chosen relayer**. The mobile picks randomly among active keepers read from this Config. An attacker watching one specific keeper only captures a subset of the global traffic.

5 System Program

INFRASTRUCTURE

11111111111111111111111111111111

The Solana System Program. Used here for two inner instructions: create the Job PDA (1419 bytes, 0.01077 SOL rent), then transfer 0.01 SOL from the submitter to the Job (fee escrow for the keeper).

Privacy role: none. This is Solana plumbing. But note that **no instruction** in this tx touches a privacy custom program directly (zk_shielded, etc). Everything flows through the p01_relayer abstraction.

6 P01 Relayer Program

PROGRAM · INVOKED

2okhzLVr6FEq5jP19KT6VurcSutx2zE4RhkRamrk5WpW

The on-chain p01_relayer program we wrote. Receives the SubmitJob ix, validates the payload format, creates the Job PDA, and emits the JobSubmittedEvent with the assigned relayer, fee, and deadline.

What it HIDES by design: nothing in this ix is in cleartext. Visible arguments are job_id (32 random bytes) plus encrypted_tx (~1.4 KB encrypted) plus routing metadata. No information about the subscriber, the merchant, the amount, or the STARK proof.

Published event: JobSubmittedEvent publishes the assigned relayer, the fee (10 M lamports = 0.01 SOL), and the deadline (slot 464,392,904). The fixed fee is fine; the deadline timing could help correlate with the keeper's second tx. Mitigation: randomized deadline in Phase E.

Total: 6 visible actors, 0 privacy custom programs invoked directly, 0 leakage of the subscriber or merchant identity. Everything flows through the **SubmitJob → Encrypted Payload** abstraction.

04 How the note was born · the shield deposit

The note we are spending today was created weeks ago by a separate shield tx. The user deposited a fixed denomination of SOL into the public pool. From that moment on, the user holds a private claim to that SOL, identified only by a Poseidon hash.

What happens in a shield tx

1 Mobile generates a fresh note locally

A 32-byte secret is sampled from a CSPRNG, then `commitment = Poseidon(secret, leaf_index, denomination)` is computed in the WebView WASM module. Secret never leaves the device, stored in Android SecureStore (PBKDF2 600K rounds).

2 Mobile signs and sends a shield tx

Calls `zk_shielded::shield_denominated` with two ixes: (a) System Program transfers N SOL from the user wallet to the pool vault, (b) `zk_shielded` receives the commitment, inserts it as the next leaf in the Merkle tree, and updates the pool root.

3 On-chain emits a LeafInserted event

The event contains the commitment plus its leaf index. Other client devices use this event to reconstruct the Merkle path later, when they need to spend their own notes.

4 After K confirmations the note is mature

The client waits ~32 slots for the Merkle root to be stable (no reorg risk). After that, the note is spendable for any private operation: unshield, transfer, or subscribe.

What is PUBLIC on-chain

- The total deposits in the pool (vault balance)
- The list of all commitments (anonymity set)
- The current Merkle root and recent history
- The fact that a wallet shielded at slot X

What stays PRIVATE

- Which commitment belongs to which wallet
- The secret behind any commitment
- What the user will do with the note later
- Any link between shield wallet and future spend wallet

The anonymity set at this stage is **every other holder of a same-denomination note**. After 100 wallets shield 1 SOL each, your commitment is indistinguishable from 99 others.

05 The keeper executes · the second tx, signed by the keeper

Within 1 to 120 seconds of the SubmitJob, the assigned keeper picks up the encrypted payload, decrypts it, and submits the real subscribe tx on-chain. This is where the STARK proof is verified by the blockchain and the fund moves to the merchant.

What happens in tx #2 (keeper-signed)

- 1 **Keeper polls its assigned Jobs queue**
Reads the Config PDA, finds the Job with its keeper PDA listed as assigned_relayer. Decrypts the payload with its ML-KEM private key. The decrypted blob is the full subscribe tx (~1.5 KB).
- 2 **Keeper submits the on-chain STARK verify (ix 1 of tx #2)**
`p01_stark_verifier::verify_stark_proof_v2` reads the previously uploaded ProofBuffer (circuit 1 = pool_commitment), runs FRI fold checks, Merkle proof checks, OOD constraints. About 1.3 M CU. On success, marks the buffer `verified=true`.
- 3 **Keeper invokes subscribe on zk_shielded (ix 2 of tx #2)**
`zk_shielded::subscribe_private_stark` reads the verified ProofBuffer, checks the public inputs include a valid pool root + nullifier, ensures the nullifier PDA does not yet exist (no replay), then creates two new accounts in one atomic ix.
- 4 **Atomic state changes on success**
Nullifier PDA created at `["nullifier", nullifier_bytes]`: prevents double-spend forever. **SubscriberVault PDA** created at `["sub_vault", merchant, commitment]`: holds (service_id, subscriber_commitment, state=Active, period_start_slot). **Pool vault → merchant fee escrow**: the note's denomination is debited from the pool and credited to the merchant.

Crucially: the keeper signs, not you

The fee payer and the sole signer of tx #2 is the keeper's PDA. On Solana Explorer, this second tx shows up as "keeper subscribed someone to merchant M". There is no on-chain graph link between your wallet (which paid for tx #1 via an ephemeral submitter) and the actual subscribe. The merchant receives the SOL from the pool vault, never from a wallet that could identify you.

The SubscriberVault stores only one field that ties to a user: `subscriber_commitment: [u8; 32]`. That commitment is the Goldilocks field element from the STARK proof. It is a hash. The merchant never sees a wallet, a name, or an email.

06 The blockchain is the medium · both sides validate without revealing

The license key is never **transmitted** between client and merchant. It is **derived** on each side from the same on-chain SubscriberVault state, by the same deterministic hash. The blockchain is the distribution channel by being a shared, immutable, public state.

How both sides derive the same key from on-chain state

```
// Identical computation, run independently by client and merchant.
license_key = BLAKE3(
  "p01-license-v1"      // domain separator
  || vault_pda          // SubscriberVault PDA, 32 opaque bytes
  || service_id         // "disney-plus" from Service Registry
)[:15]                 // 120 bits of entropy

display = "P01-" + crockford_base32(license_key).match(/.{4}/g).join("-")
// → "P01-XQ8B-3F2A-K7M5-9PRD-WCNT-Y4ZH"
```

What each side validates · what each side reveals (nothing)

Client side

- Reads the SubscriberVault PDA from Solana (state, period bounds)
- Re-derives the key on every render — no local storage
- Shows it in the LicenseKeyCard component (shipped 2026-05-23, apps/mobile/components)
- **Reveals nothing**: the displayed string is opaque without the vault PDA and the service salt

Merchant side

- Scans `getProgramAccounts(zk_shielded)` filtered by their merchant pubkey
- For each active SubscriberVault, recomputes the expected key
- Matches against the presented key ($O(N)$ over active subs, < 50 ms typical)
- **Reveals nothing**: no oracle calls, no off-chain DB of subscribers, no PII handling

SDK · the merchant integration in one line

```
import { hasAccess } from '@protocol01/specter-sdk';

// On every authenticated API call from the client:
const ok = await hasAccess(commitment, 'disney-plus');
if (!ok) return res.status(402).json({ error: 'subscription required' });
return res.json({ /* content */ });
```

What hasAccess does — pure on-chain, no tokens to issue

- **Single on-chain read**: `connection.getAccountInfo(SubscriberVault_PDA)`, derived from `[merchant_pubkey, commitment, service_id]`. Returns the vault, or null.
- **Decision**: returns **true** if `vault.state === Active` and the current slot is inside an active period. Else **false**.
- **No token issuance**: the SubscriberVault PDA **is** the credential. No JWT to sign, no session cookie to store, no off-chain database to maintain. The merchant's "session" lives entirely on Solana.
- **Optional cache**: for perf, the merchant can cache (`commitment → vault.state`) in Redis with a 10-30

07 Active · paused · expired · deleted

A subscription transitions through a four-state machine on-chain. The merchant's "ephemeral account" mirrors that state automatically — frozen on pause, deleted on expiry — without the merchant ever learning who the user is.

State transitions · what each one triggers

FROM → TO	TRIGGER	ON-CHAIN EFFECT	MERCHANT EPHEMERAL ACCOUNT
Active → Paused	Client signs <code>pause_private_stark</code> with a fresh circuit-0 STARK proof of ownership.	<code>Vault.state = Paused</code> . Pause slot stored to track elapsed time across the freeze.	Next <code>hasAccess</code> read returns false . Access frozen instantly the moment the on-chain tx confirms. Any cached "active" entry on the merchant side ages out within 10-30 slots.
Paused → Active	Client signs <code>resume_private_stark</code> with a fresh STARK proof.	<code>Vault.state = Active</code> . The period boundary rolls forward by the pause duration so the user gets back the time they paused.	Next <code>hasAccess</code> read returns true again. Access resumes seamlessly on the very next API call.
Active → Expired	Note balance < next-period rate at the period boundary. Keeper detects and calls <code>close_expired_vault</code> .	Vault closed, PDA deleted. Residual rent refunded to a fresh shielded note (anonymity preserved). Last paid period fully honored before close.	Next <code>hasAccess</code> read returns null (vault not found) → access refused. User invited to re-subscribe with a new note.
Active → Cancelled	Client signs <code>cancel_private_stark</code> with a fresh STARK proof.	Residual lamports routed via <code>p01_relayer RefundJob</code> as a new shielded note. Vault closed in the same atomic flow.	Same as Expired: <code>hasAccess</code> returns null → access refused.

The merchant's "ephemeral account" is the vault PDA itself

```
# The merchant has no users table, no sessions table, no JWT signing key.
# Their only off-chain state is an optional cache, for performance only.

cache: dict[Commitment, VaultState] = {} # Redis or in-memory, TTL ~10-30 slots

def hasAccess(commitment: bytes, service_id: str) -> bool:
    vault = cache.get(commitment) or chain.read_vault(
        SubscriberVault_PDA(merchant_pubkey, commitment, service_id)
    )
    if vault is None: return False # Vault closed → access refused
    if vault.state != 'Active': return False # Paused/cancelled → frozen
    if now_slot() >= vault.period_end: return False # Period expired
    return True

# NO name. NO email. NO card. NO wallet. NO IP. NO JWT. NO database.
# A merchant breach leaks: an optional cache of public on-chain state.
# Worth nothing.
```

The expiry path: no renewal, no PII to delete

When the underlying note runs out of funds and the subscriber chose **no** auto-renewal, the keeper's cron

08 Who hides what · summary by data point

Each row is a data point an attacker would want to correlate. Each column is an actor of the tx. Green means hidden, red means leak, yellow means partially hidden.

DATA POINT	SUBMITTER	JOB PDA	KEEPER	PROGRAM	NATIVE CRYPTO
User wallet	✓ broken	—	—	—	✓ stealth
Merchant identity	—	✓ encrypted	✗ sees cleartext	✓ never on-chain in clear	—
Spent note (which one)	—	✓ encrypted	✗ sees cleartext	—	✓ nullifier + STARK ownership
Note amount	—	✓ encrypted	✗ sees cleartext	—	✓ denominated pool, anonymity set N
Spending key	—	—	—	—	✓ never leaves the device
STARK proof (80 KB)	—	✓ encrypted in payload	✗ sees cleartext	—	✓ zero-knowledge (no witness leak)
IP / network metadata	✓ mobile RPC is the client	—	~ keeper sees submitter IP (but ephemeral)	—	—
Subscribe timing	—	—	✓ queue introduces a 1-120 s delay	—	—
Wallet → subscribe graph	✓ 1 ephemeral hop	—	✓ 2nd tx signed by keeper	—	✓ uncorrelated nullifier

Residual trust assumptions

Trust 1 · The keeper

The assigned keeper sees the payload in cleartext for a few ms. Mitigation: multi-relayer registry (mobile picks among N keepers), automatic failover, and roadmap TEE (Intel SGX/TDX) so even the keeper never sees the cleartext.

Trust 2 · Solana validators

Same assumption as for any Solana tx. PoS, ~3000 active validators, supermajority finality. No extra trust specific to Protocol 01.

VS Umbra (Arcium MPC + Groth16): two trust assumptions on their side too, but heavier. (1) Arcium's MPC committee: if $t+1$ nodes collude, the cleartext leaks (and the nodes are identified on-chain). (2) Groth16's trusted setup: the toxic waste of the ceremony compromises everything. On our side: transparent STARK (no ceremony) plus Goldilocks/Poseidon (no elliptic curves, therefore Shor-resistant).